

12 Requerimientos Avanzados para Examen en C (Alta Dificultad)

1. Compresor de Matrices por Segmentos con Recursividad y Punteros

Desarrolle un programa que:

- Lea una matriz de enteros de tamaño dinámico.
- Implemente una función recursiva que identifique segmentos consecutivos iguales en cada fila y los “comprime” en pares (valor, cantidad).
- La compresión debe almacenarse dinámicamente usando punteros.
- El programa debe reconstruir la matriz original a partir de la compresión para validar que el proceso es correcto.
- Mostrar:
 - Matriz original
 - Estructura comprimida
 - Matriz reconstruida

2. Sistema de Rotación Multinivel de Matrices con Validación Compleja

Construya un programa que:

- Genere una matriz cuadrada con valores aleatorios.
- Implemente una función que rote la matriz en niveles concéntricos (anillos), donde cada nivel se rota en sentido contrario al anterior.
- La rotación debe realizarse mediante punteros y aritmética de direcciones, sin usar matrices auxiliares.
- Utilice condicionales para validar:
 - Si el número de niveles es par o impar
 - Si la rotación produce una matriz palindrómica por filas o columnas

- Mostrar la matriz antes y después de la rotación.

3. Árbol de Decisiones Simulado con Matrices y Recursividad

El programa debe:

- Representar un árbol de decisiones usando una matriz donde cada celda contiene un valor que indica la “decisión” siguiente.
- Implementar una función recursiva que simule el recorrido del árbol desde una posición inicial hasta llegar a un nodo terminal.
- El recorrido debe detenerse si:
 - Se detecta un ciclo
 - Se sale de los límites
 - Se repite una celda
- Utilizar punteros para manipular la matriz dentro de las funciones.
- Mostrar:
 - Camino recorrido
 - Número de decisiones tomadas
 - Si el recorrido fue válido o fallido

4. Generador de Permutaciones con Restricciones Matriciales

Desarrolle un programa que:

- Lea un conjunto de números y los almacene en un arreglo dinámico.
- Genere todas las permutaciones posibles mediante una función recursiva.
- Cada permutación debe insertarse en una matriz dinámica donde cada fila representa una permutación.
- Solo deben almacenarse las permutaciones que cumplan una regla matricial, por ejemplo:

- La suma de los elementos en posiciones pares debe ser mayor que la suma de las posiciones impares
- Utilizar punteros para gestionar la matriz de permutaciones.
- Mostrar:
 - Cantidad total de permutaciones generadas
 - Cantidad de permutaciones válidas
 - La permutación válida con mayor suma total

5. Sistema de Evaluación de Expresiones con Punteros y Matrices de Prioridad

Construya un programa que:

- Lea una expresión matemática como cadena.
- Convierta la expresión en una matriz donde cada fila representa un token (número u operador) y su prioridad.
- Implemente una función recursiva que evalúe la expresión respetando las prioridades, sin usar eval, stack explícito ni librerías externas.
- Utilice punteros para recorrer y modificar la matriz de tokens.
- Mostrar:
 - Matriz de tokens
 - Resultado final
 - Pasos intermedios de la evaluación

6. Sistema de Simulación de Ecosistema con Matriz Dinámica y Reglas Complejas

El programa debe:

- Crear una matriz dinámica que represente un ecosistema con tres tipos de entidades:
 - 0 = vacío
 - 1 = presa
 - 2 = depredador

- Implementar una función recursiva que simule la evolución del ecosistema por turnos, aplicando reglas como:
 - Una presa se reproduce si tiene al menos dos presas vecinas
 - Un depredador muere si no encuentra presa en dos turnos
 - Un depredador se mueve hacia la presa más cercana
- Utilizar punteros para actualizar la matriz sin crear una copia completa.
- Mostrar:
 - Estado inicial
 - Estado final después de turnos
 - Estadísticas de población

1. Analizador de Matriz con Transformaciones Recursivas y Punteros

Desarrolle un programa en C que:

- Lea dinámicamente una matriz cuadrada de tamaño utilizando punteros dobles.
- Implemente una **función recursiva** que genere la **matriz transpuesta**, sin crear una matriz auxiliar completa; solo debe intercambiar elementos mediante aritmética de punteros.
- Aplique condicionales para determinar si la matriz original es:
 - Ortogonal
 - De rango completo
- Muestre un informe final comparando la matriz original y la transpuesta, indicando todas las propiedades detectadas.

2. Resolución de un Laberinto con Restricciones y Backtracking Avanzado

Implemente un programa que:

- Cargue un laberinto desde un archivo de texto en una matriz de caracteres.
- Utilice una **función recursiva con backtracking** que encuentre **todos los caminos posibles** desde la entrada hasta la salida.
- Cada movimiento debe cumplir una regla adicional:
 - No se puede pasar por más de dos celdas consecutivas con el mismo símbolo.
- Al finalizar, el programa debe:
 - Mostrar la cantidad total de caminos encontrados
 - Imprimir el camino más corto y el más largo
 - Indicar si existe un camino que pase por todas las esquinas del laberinto

3. Sistema de Gestión de Datos con Matriz de Estructuras y Búsqueda Recursiva

Construya un programa que:

- Genere dinámicamente una matriz de estructuras que representen registros (ID, nombre, valor numérico).
- Implemente funciones para:
 - Ordenar la matriz completa por ID usando **punteros y un algoritmo recursivo**
 - Buscar un registro por nombre mediante **búsqueda binaria recursiva**
- Utilice condicionales para validar:
 - IDs duplicados
 - Nombres vacíos
 - Valores fuera de rango
- Muestre un reporte final con estadísticas (mínimo, máximo, promedio), calculadas mediante punteros.

4. Generador de Submatrices y Determinantes con Recursividad Compleja

El programa debe:

- Leer una matriz cuadrada de tamaño .
- Generar todas las **submatrices de orden** utilizando punteros y sin crear estructuras auxiliares completas.
- Calcular el **determinante de cada submatriz** mediante una función recursiva basada en cofactores.
- Identificar cuál submatriz tiene el determinante:
 - Mayor
 - Menor
 - Más cercano a cero
- Mostrar un informe final con los resultados.

5. Simulador de Procesamiento Numérico con Punteros y Funciones de Alto Nivel

Desarrolle un programa que:

- Cree dinámicamente un arreglo de tamaño variable usando punteros.
- Transforme el arreglo en una matriz mediante aritmética de punteros.
- Implemente una función recursiva que:
 - Clasifique los números en tres categorías: primos, compuestos y especiales (perfectos, abundantes o deficientes).
- Utilice condicionales y ciclos para generar un **mapa de calor textual**, donde cada categoría se represente con un símbolo distinto.
- Muestre estadísticas avanzadas:
 - Frecuencia por categoría
 - Promedio por fila y columna
 - Elemento más repetido

6. Sistema de Cifrado y Descifrado con Matrices, Recursividad y Punteros

El programa debe:

- Leer una cadena y almacenarla en una matriz de caracteres de tamaño dinámico.
- Implementar un cifrado por bloques, donde cada bloque se invierte mediante una función recursiva.
- Reorganizar la matriz aplicando un patrón en zigzag usando punteros.
- Validar mediante condicionales:
 - Longitud mínima
 - Caracteres no permitidos
- Mostrar:
 - Texto cifrado
 - Texto descifrado
 - Comparación entre ambos procesos